

后端第3题

Task 1

```
// 文件名: Main.java (public 类名要和文件名一致)

public class Main {

    // main 方法: 程序入口
    public static void main(String[] args) {

        // 1. 用普通构造方法创建对象, 并赋值
        Person p1 = new Person("张三", 20, 1);

        // 2. 用复制构造方法, 复制出一个新对象
        Person p2 = new Person(p1);

        // 3. 打印两个对象的信息, 验证复制成功
        p1.introduce();    // 大家好, 我叫张三
        p2.introduce();    // 大家好, 我叫张三 (值一样)
    }
}

// Person 类 (可以写在同一个文件里, 但不能加 public)
class Person {
    private String name;
    private int age;
    private int sex;    // 1=男, 0=女 (这里约定)

    // 普通构造方法
    public Person(String name, int age, int sex) {
        this.name = name;
        this.age = age;
        this.sex = sex;
    }

    // 复制构造方法 (实现“复制对象”)
}
```

```

public Person(Person other) {
    this.name = other.name;
    this.age = other.age;
    this.sex = other.sex;
}

// 打印信息的方法（方便验证）
public void introduce() {
    System.out.println("大家好，我叫" + name + "，今年" + age + "岁");
}
}

```

- 用到了this，this作用：避免出现重名冲突「构造方法的参数叫 name（小括号里的），类的字段也叫 name（类顶部声明的）」用了this就给java指明我说的是这个对象自己的字段.一句话总结 this 的作用：this 代表"当前这个对象"。当字段名和参数名重名时，用 this.字段名 明确指向"对象的字段"，避免混淆。

	类 (class)	对象 (object)
是什么	一个 模板/图纸/模具	用模板造出来的 具体实物
有几个	通常一个	可以造出无数个
能不能直接"用"	不能直接吃（模具不能吃）	能（饼干能吃）
例子	<code>Person</code> （人的模板）	张三、李四（具体的人）

- 类是对象的模板/图纸，描述"某一类东西有哪些属性和行为"；对象是类的具体实例，是根据类创建出来的真实存在的个体。一个类可以创建多个对象

Task 2

```

public class Main {
    public static void main(String[] args) {
        Person p1 = new Person("张三", 20, 1);
        Person p2 = new Person("李四", 21, 0);

        // 调用实例方法（需要对象）
        p1.introduce();

        // 调用静态方法（直接用类名）
        System.out.println("一共创建了 " + Person.getCount() + " 个人");
    }
}

```

```

}
}

class Person {
    private String name;        // private: 只有类内部能访问
    private int age;
    private int sex;

    private static int count = 0;    // 静态变量，统计对象数量

    public Person(String name, int age, int sex) {
        this.name = name;
        this.age = age;
        this.sex = sex;
        count++;
    }

    // 实例方法（需要对象，因为要用 name、age）
    public void introduce() {
        System.out.println("我叫" + name + "，今年" + age + "岁");
    }

    // 静态方法（属于类，可直接类名调用）
    public static int getCount() {
        return count;
    }
}

```

修饰符	中文	我的记忆方法	谁能访问
<code>public</code>	公开	公共广场	所有人
<code>protected</code>	受保护	家族内部+子女	自己、同包、子类
默认（什么都不写）	默认	同小区	自己、同包
<code>private</code>	私有	私人卧室	只有自己

public（最开放） > protected > 默认 > private（最封闭）

- 因为静态方法属于类，不属于任何具体对象。它不需要"先造一个对象"就能存在，就像"教室的黑板"不需要某个学生来就能存在。所以直接 类名.方法名() 就能调
- 因为实例方法里通常要用到对象的字段（比如 name、age），而这些字段是"每个对象各自的"。没有对象，这些字段就不存在，方法自然没法运行。所以必须先 new 出对象，才能调用实例方法

Task 3

```
public class Main {
    public static void main(String[] args) {
        Animal penguin = new Penguin("小企", 1);
        Animal mouse = new Mouse("小杰瑞", 2);

        penguin.eat();           // 企鹅小企正在吃小鱼
        penguin.introduction(); // 大家好！我是1号小企。

        mouse.eat();             // 老鼠小杰瑞正在吃奶酪
        mouse.introduction();    // 大家好！我是2号小杰瑞。
    }
}

// 父类 Animal
class Animal {
    private String name; // private: 子类不能直接访问，要通过 super 让父类赋值
    private int id;

    public Animal(String myName, int myId) {
        name = myName;
        id = myId;
    }

    public void eat() {
        System.out.println(name + "正在吃");
    }

    public void sleep() {
        System.out.println(name + "正在睡");
    }

    public void introduction() {
```

```

        System.out.println("大家好！我是" + id + "号" + name + "。");
    }
}

// 子类 Penguin
class Penguin extends Animal {
    public Penguin(String myName, int myId) {
        super(myName, myId);    // super 调父类构造方法
    }

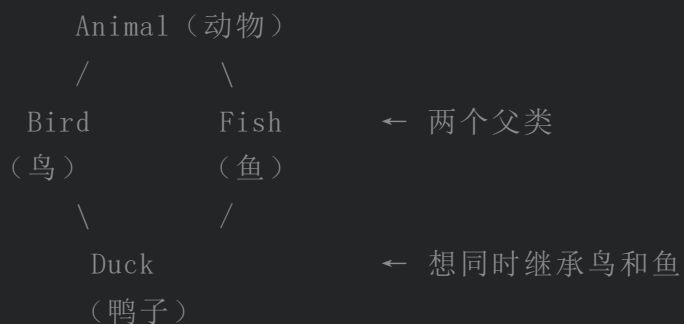
    @Override
    public void eat() {
        System.out.println("企鹅" + name + "正在吃小鱼");
    }
}

// 子类 Mouse
class Mouse extends Animal {
    public Mouse(String myName, int myId) {
        super(myName, myId);
    }

    @Override
    public void eat() {
        System.out.println("老鼠" + name + "正在吃奶酪");
    }
}

```

菱形冲突



如果 Bird 和 Fish 都有一个 move() 方法：

Bird 的 move(): 飞

Fish 的 move(): 游

那么 Duck（鸭子）继承了两个 `move()`，当调用 `duck.move()` 时，Java 就懵了：到底用飞的还是

Java 的解决办法：干脆禁止类多继承，从根源上消灭这种歧义

Task 4

```
public class Main {
    public static void main(String[] args) {
        Animal penguin = new Penguin("小企", 1);
        Animal mouse = new Mouse("小杰瑞", 2);

        penguin.eat();           // 企鹅小企正在吃小鱼
        penguin.introduction();  // 大家好！我是1号小企。

        mouse.eat();             // 老鼠小杰瑞正在吃奶酪
        mouse.introduction();    // 大家好！我是2号小杰瑞。
    }
}

// 父类 Animal
class Animal {
    private String name;  // 字段保持 private，保护起来
    private int id;

    public Animal(String myName, int myId) {
        name = myName;
        id = myId;
    }

    // 新增：getter 方法，让子类 and 外部能“安全地”读 name
    public String getName() {
        return name;
    }

    public void eat() {
        System.out.println(name + "正在吃");
    }

    public void sleep() {
```

```

        System.out.println(name + "正在睡");
    }

    public void introduction() {
        System.out.println("大家好！我是" + id + "号" + name + "。");
    }
}

// 子类 Penguin
class Penguin extends Animal {
    public Penguin(String myName, int myId) {
        super(myName, myId);    // super 调父类构造方法
    }

    @Override
    public void eat() {
        // 不再直接写 name，而是用 getName() 方法去读
        System.out.println("企鹅" + getName() + "正在吃小鱼");
    }
}

// 子类 Mouse
class Mouse extends Animal {
    public Mouse(String myName, int myId) {
        super(myName, myId);
    }

    @Override
    public void eat() {
        System.out.println("老鼠" + getName() + "正在吃奶酪");
    }
}

```

```

public interface Shape {
    double area();
    double perimeter();
}

class Circle implements Shape {    // 注意是 implements，不是 extends
    private double radius;
}

```

```

public Circle(double radius) { this.radius = radius; }

@Override
public double area() { return Math.PI * radius * radius; }

@Override
public double perimeter() { return 2 * Math.PI * radius; }
}
// Rectangle、Triangle 类似，把 extends 改成 implements 即可

```

关键区别：class Circle extends Shape（抽象类）→ class Circle implements Shape（接口）

Task 5

```

public class BankAccount {
    // 字段全部改成 private，保护内部数据
    private String accountNumber;    // 账号
    private String accountHolder;    // 户主名
    private double balance;          // 余额
    private String password;         // 密码

    // 构造方法：初始化账户
    public BankAccount(String accountNumber, String accountHolder,
                        double initialBalance, String password) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
        this.password = password;
    }

    // 存款（不需要密码，存钱谁都能存）
    public void deposit(double amount) {
        if (validateAmount(amount)) {    // 校验金额是否合法
            balance += amount;           // 余额增加
            System.out.println("存款成功，当前余额：" + balance);
        } else {
            System.out.println("存款失败：金额必须大于0");
        }
    }
}

```

```
// 取款（需要密码）
public boolean withdraw(double amount, String inputPassword) {
    // 先校验密码
    if (!validatePassword(inputPassword)) {
        System.out.println("取款失败：密码错误");
        return false;
    }
    // 再校验金额合法 + 余额充足
    if (!validateAmount(amount) || amount > balance) {
        System.out.println("取款失败：金额非法或余额不足");
        return false;
    }
    balance -= amount;    // 扣钱
    System.out.println("取款成功，当前余额：" + balance);
    return true;
}

// 转账（需要密码）
public boolean transfer(BankAccount recipient, double amount, String inputPassword) {
    if (!validatePassword(inputPassword)) {
        System.out.println("转账失败：密码错误");
        return false;
    }
    if (!validateAmount(amount) || amount > balance) {
        System.out.println("转账失败：金额非法或余额不足");
        return false;
    }
    // 自己扣钱，对方加钱
    balance -= amount;
    recipient.balance += amount;
    System.out.println("转账成功，已转出 " + amount + " 元");
    return true;
}

// 查询余额（只给看，不给直接改）
public double getBalance() {
    return balance;
}

// 获取账户信息（敏感信息脱敏，不暴露密码）
public String getAccountInfo() {
```

```

        // 账号只显示后4位，隐藏密码，保护隐私
        String masked = accountNumber.substring(accountNumber.length() - 4);
        return "户主：" + accountHolder + "，账号尾号：" + masked;
    }

    // 校验密码（改成 private，只供类内部使用）
    private boolean validatePassword(String inputPassword) {
        return password.equals(inputPassword);
    }

    // 校验金额（改成 private，只供类内部使用）
    private boolean validateAmount(double amount) {
        return amount > 0;    // 金额必须大于0，不能是负数或0
    }
}

```

测试

```

public class Main {
    public static void main(String[] args) {
        BankAccount a = new BankAccount("12345678", "张三", 1000, "123456");
        BankAccount b = new BankAccount("87654321", "李四", 500, "654321");

        a.deposit(200);                // 存款成功，余额1200
        a.withdraw(300, "123456");      // 取款成功，余额900
        a.withdraw(300, "wrong");       // 密码错误
        a.transfer(b, 200, "123456");  // 转账成功
        System.out.println("A余额：" + a.getBalance());    // 700
        System.out.println("B余额：" + b.getBalance());    // 700
        System.out.println(a.getAccountInfo());    // 只显示尾号，不暴露密码
    }
}

```

运行结果在库里